C Code Visualizer

A Real-Time, Web-Based C Program Visualization Tool

Ganesh Bharate (23189) · Kuldeep Jatav (23684)

Introduction to Algorithms and Software Programming · 28 April 2026

github.com/Kuldeep1709/C-Code-Visualizer

Motivation

The Problem

1. Standard code editors and debuggers are powerful tools for code development, yet lack a simple UI.
2. Most of them also do not possess backward step or history tracking.
3. Using debuggers do require thorough understanding of program flow and significant experience.

Our Solution

- ✔ It allows editing the C code directly in browser - no installation required.
- ✔ It executes the code line by line.
- ✔ A live display of local variables, types, values & call stack panels is maintained throughout.
- ✔ A step back feature is introduced by which user can replay any prior execution state instantly.

System Workflow

1. **Code Entry:** User writes C code in the frontend `CodeEditor`.
2. **Compilation Request:** Frontend sends HTTP POST request to `/api/compile`.
3. **Engine Instantiation:** Python server creates session ID, temp directory, and C++ engine subprocess.
4. **Compilation via Engine:** Server sends JSON command `{"cmd":"compile"}`; engine invokes `gcc/g++ -g`.
5. **WebSocket Connection:** Frontend connects to `/ws/{session_id}` after successful compilation.
6. **Debugging Loop:** Commands `step`, `next`, `continue` flow through server to GDB via the C++ engine.
7. **State Extraction:** Engine parses GDB-MI output into JSON: current line, variables, call stack, stdout.
8. **UI Update:** `useDebugSession` updates state, stores history, and re-renders UI panels.

Technology Stack

Frontend

React + TypeScript + Vite

Component-based UI; fast dev server & bundling

Debugger

GDB Machine Interface (MI)

-exec-next, -exec-step, -stack-list-frames commands

Middleware

Python FastAPI + Uvicorn

Async REST & WebSocket server; thin bridge layer

Build

CMake

Compiles the `visualizer` engine binary

Engine

C++ (g++ -g)

Process control, GDB management, type-safe state structs

Transport

JSON over HTTP + WebSocket

All inter-component messaging; structured & typed

UI Features

CodeEditor.tsx

Code editor with execution line highlighted

ControlPanel.tsx

Start, Stop, Step, Continue, Step Back.

VariablePanel.tsx

Displays variables: name, type, value.

StackPanel.tsx

Shows call stack frames and line info.

ConsolePanel.tsx

Displays stdout and stderr output.

Real Time Demo

Limitations and Future Scope

- Breakpoint support
- `scanf()` functionality or Runtime Input Feature.
- Heap / dynamic memory allocation visualization
- Pointer arrow visualization
- Deploying

Thank You!

Questions & Discussion

github.com/Kuldeep1709/C-Code-Visualizer

Ganesh Bharate (23189) · Kuldeep Jatav (23684)